Coding & Solution

Date	2 July 2026
Team ID	
Project Name	Smart Lender
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/pavanikaranam-sys/smartbridge-smartlender
Programming Language(s)	Python
Framework(s) Used	Flask; scikit-learn and XGBoost for modeling
Key Features Implemented	• Data preprocessing pipeline: null-value checks, dropping non-predictive Loan_ID column, and Label Encoding of categorical fields (Gender, Married, Dependents, Education, Self_Employed, Property_Area, Loan_Status) • Training and comparison of four classification algorithms — Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost — for loan default/repayment prediction • Hyperparameter tuning of the Random Forest model using GridSearchCV (cross-validated search over n_estimators, max_depth, min_samples_split, min_samples_leaf) • Model selection based on comparative accuracy: XGBoost identified as the best performer (94.7% training accuracy, 81.1% testing accuracy) • Model persistence using joblib for serializing the trained model (.pkl) for reuse in the web application • Flask-based web application for real-time loan approval prediction from applicant inputs • User interface allowing applicants to submit gender, marital status,

	education, employment status, income, loan amount, loan term, credit history, and property area details and instantly receive a prediction • Solution architected for deployment on IBM Cloud
Pending / Incomplete Features	• End-to-end integration testing of the Flask application with the finalized XGBoost model (current notebook trains and evaluates the models; the production model artifact and Flask routes should be verified together before release) • Deployment configuration and validation on IBM Cloud • Input validation and error handling on the web form for malformed or missing applicant data • Batch-upload / bulk evaluation feature referenced in the high-volume analyst use case
Setup / Run Instructions	1. Create a Python environment (Anaconda recommended) with Python 3.8+. 2. Install dependencies: pip install numpy pandas scikit-learn xgboost matplotlib seaborn scipy flask joblib 3. Run the training notebook (smart_lender.ipynb) to preprocess the data and generate the trained model file (loan_prediction_model.pkl). 4. Place the saved model file in the Flask application directory and start the server: python app.py 5. Open a browser at http://localhost:5000 to access the loan prediction interface.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Partial
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	No
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	

Additional Notes / Comments:

The evaluation above is based on the model-training notebook (smart_lender.ipynb) provided, which covers data preprocessing, model comparison (Decision Tree, Random Forest, KNN, XGBoost), hyperparameter tuning, and model serialization. The Flask front-end/back-end source files were not included in the reviewed materials; once added, this checklist should be revisited for items 1, 4, and 6.